High-Level Architecture Overview

What we've built is a **serverless facial recognition system on AWS** with this core purpose:

- **Register employees (store faces)**
- **Authenticate visitors/employees (compare faces)**

It follows a **fully event-driven + API-driven architecture**.

1. Actors (Left Side of Diagram)

- **Employee (top left)** → uploads image for registration
- **Visitor/Employee (bottom left)** → submits image for authentication

These users interact through the **React frontend**

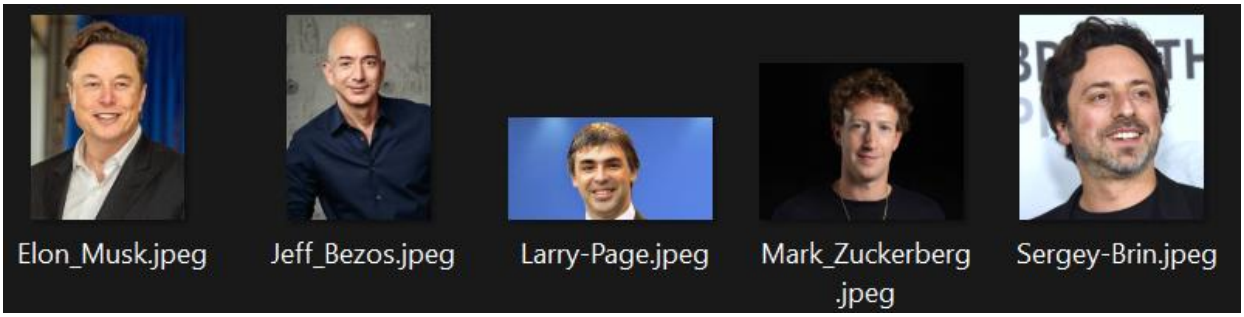
2. Frontend Layer (React App)

- Runs locally (`localhost:5173`) and provides:
  - Upload form (First name, Last name, image)
  - Preview + response display (success/failure)

Seen clearly in:

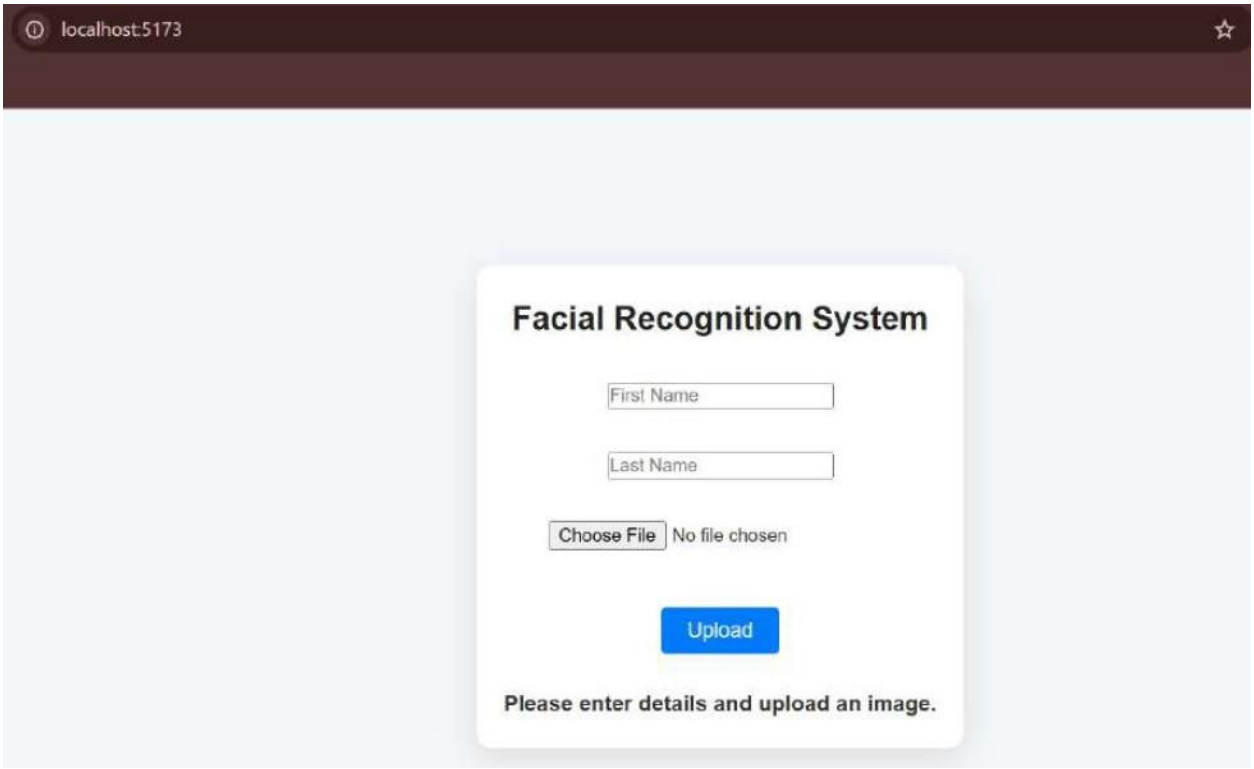
- UI screenshots (form + upload button)
- Success/Failure messages

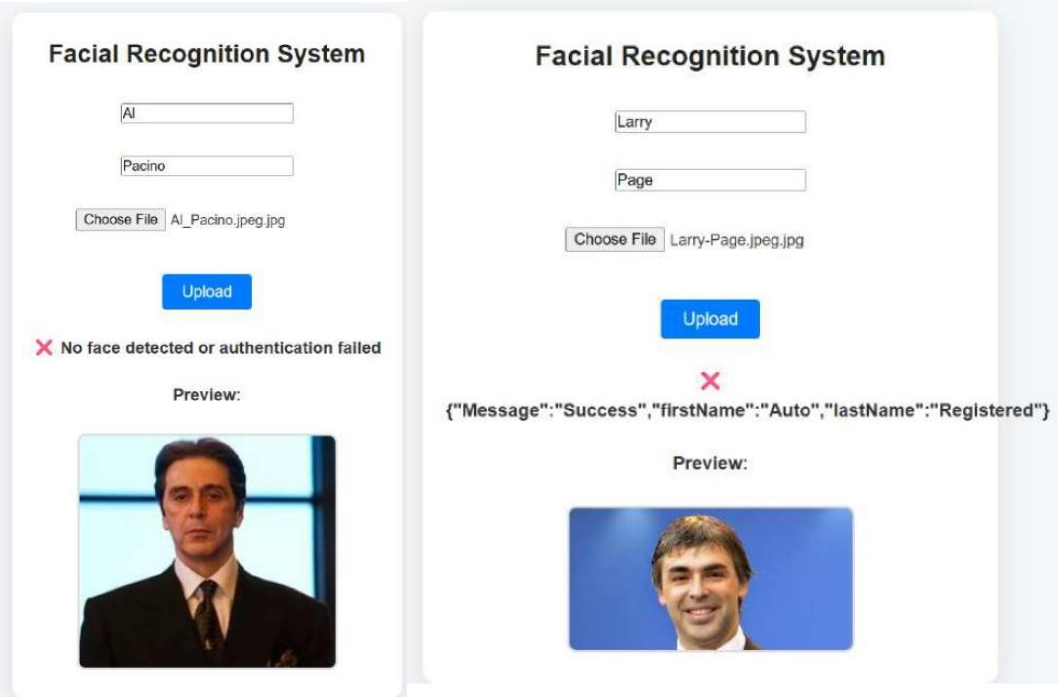
Employee-images



Visitor-images


al-pacino-2
kevin-spacey-2
nicolas-cage-2
ryan-gosling-2
sean-connery-2


localhost:5173
Facial Recognition System
First Name
Last Name
Choose File
No file chosen
Upload
Please enter details and upload an image.



Role in architecture:

- Sends HTTP requests → API Gateway
- Displays response from backend (Lambda)

3. API Layer (API Gateway)

From API Gateway PDF:

- REST API: `facial-recognition-api`
- Endpoint:

  `/bucket (POST)`

- Deployed to stage `v1`

Evidence:

- Resource `/bucket` with POST method
- Lambda integration configured

What it does:

- Acts as **entry point to backend**
- Routes requests to:
  - **Auth Lambda**

4. Core Compute Layer (Lambda Functions)

We have **two key Lambda functions**:

A. Registration Lambda

From diagram: **Top Lambda (Registration Lambda)**

- Function: `employee-registration`
- Trigger: **S3 (employee-images bucket)**

Flow:

1. Employee uploads image → S3
2. S3 triggers Lambda
3. Lambda:
    - Calls Rekognition
    - Extracts face features
    - Stores metadata in DynamoDB

B. Authentication Lambda

From diagram: **Bottom Lambda (Auth Lambda)**

- Function: `employee-authentication`
- Triggered via **API Gateway**

Flow:

1. User uploads image via frontend
2. API Gateway → Lambda
3. Lambda:
    - Fetches stored faces
    - Calls Rekognition `compare_faces`
    - Returns match result

5. Storage Layer (S3 Buckets)

We created **two buckets**:

Employee Images Bucket

- `samuel-employee-images`
- Stores **registered employee faces**

Used by:

- Registration Lambda (trigger source)

Visitor Images Bucket

- `samuel-visitor-images`
- Stores **incoming images for authentication**

Used by:

- Auth Lambda (reads images)

6. AI Layer (Amazon Rekognition)

Central component in diagram

CLI proof:

```
aws rekognition create-collection \
--collection-id employees \
--region eu-west-1
```

✓ Collection created successfully (FaceModelVersion 7.0)

What Rekognition does:

During Registration:

- Detects face
- Stores face vector in **collection**

During Authentication:

- Compares:

  ```
  employee face (collection)
  vs
  visitor image
  ```

- Returns similarity score

7. Database Layer (DynamoDB)

- Table: `employee`
- Primary key: `rekognitionid`

Contains:

- Face ID (from Rekognition)
- First name
- Last name
- Image key

Role:

- Maps:

  ```
  FaceID → Human Identity
  ```

Without this:

- Rekognition only knows faces
- DynamoDB gives them meaning

8. IAM (Security Layer)

We created roles for:

- **Lambda execution**
- **API Gateway logging**

Permissions include:

- S3 access
- Rekognition access
- DynamoDB access
- CloudWatch logging

Why this matters:

Each service operates with **least privilege access**

9. Observability (CloudWatch)

- Logs for Lambda executions
- Shows:
  - Request IDs
  - Duration
  - Memory usage

- Execution status

Role:

- Debugging
- Monitoring
- Performance tracking

10. End-to-End Data Flow

Now let's tie everything together exactly as the diagram shows:

FLOW 1: Employee Registration

1. Employee uploads image → S3 (employee bucket)
2. S3 triggers **Registration Lambda**
3. Lambda:
   - Sends image → Rekognition
   - Stores face in **collection**
   - Stores metadata → DynamoDB

✓ Outcome:

Employee is now "known" to the system

FLOW 2: Authentication

1. User uploads image via React UI
2. React → API Gateway
3. API Gateway → Auth Lambda
4. Lambda:
   - Gets image from S3
   - Calls Rekognition compare
   - Matches against stored faces
   - Fetches identity from DynamoDB
5. Response returned to frontend:
   - ✅ Success → user recognised
   - ❌ Failure → no match

Key Architecture Strengths

Fully Serverless

- No EC2
- Auto scaling
- Pay-per-use

Event-Driven + API Hybrid

- S3 triggers (registration)
- API Gateway triggers (authentication)

Decoupled Design

Each component is independent:

- S3 → storage
- Lambda → logic
- Rekognition → AI
- DynamoDB → identity

Highly Scalable

- S3 infinite storage
- Lambda auto-scale
- DynamoDB on-demand

Summary

Our architecture is:

> **A serverless facial recognition system where images are stored in S3, processed by Lambda, analysed by Rekognition, and mapped to identities in DynamoDB - all exposed via API Gateway and consumed by a React frontend.**